

An Energy-Efficient LoRa-Based Smart Weather Station with Edge AI Integration

Hoang Duc Thinh, Huynh Nhat, Thai Thanh Long, Dao Chi Bang
Hoang Duc Thinh – Lead Hardware & Firmware | Huynh Nhat – Lead AI & Cloud
Thai Thanh Long – Team Member | Dao Chi Bang – Team Member
FPT University, Vietnam

Abstract—The growing trend toward smart agriculture and precision environmental monitoring calls for reliable, low-power, long-range meteorological data acquisition systems. Traditional weather stations typically rely on cellular (GSM) or Wi-Fi networks, which are severely constrained when deployed in remote rural areas due to high power consumption and poor signal coverage. This report presents a highly optimized, distributed smart weather station that adopts a unified "ESP-Only" microcontroller architecture combined with a Long Range (LoRa) SX1278 transceiver module. By completely eliminating redundant hardware such as an Arduino UNO, the proposed design significantly reduces hardware footprint, Bill of Materials (BOM) cost, and improves energy efficiency. The completed system integrates environmental sensors (DHT11, BMP280, and an analog rain sensor), a Deep Sleep power-saving mechanism, and a secure LoRa Peer-to-Peer (P2P) communication link based on a custom 15-byte packed binary payload. At the receiving end, a Gateway Node concurrently hosts a local Edge AI Web Dashboard – enabling zero-latency monitoring and early-warning alerts independent of Internet connectivity – while asynchronously synchronizing telemetry data to the ThingSpeak cloud platform and integrating a machine-learning forecasting service. Field test results confirm that the system operates reliably, conserves energy, and delivers trustworthy weather information both with and without Internet access.

Index Terms—Internet of Things (IoT), LoRa, ESP8266, Precision Agriculture, Edge AI, Weather Station, Edge Computing.

I. INTRODUCTION AND PROBLEM STATEMENT

Continuous monitoring of meteorological parameters such as temperature, relative humidity, atmospheric pressure, and rainfall is of critical importance for precision agriculture, disaster management, and climate change research. With the ongoing expansion of the Internet of Things (IoT) ecosystem, numerous remote-sensing architectures have been proposed. However, a fundamental challenge remains in deploying weather stations in remote agricultural areas: the trade-off between communication range, power consumption, and network autonomy.

Traditional IoT weather stations commonly rely on Wi-Fi (IEEE 802.11) or GSM/GPRS networks for data transmission. While these technologies offer high data throughput and direct Internet connectivity, they consume substantial power and provide limited coverage in rural regions. Wi-Fi requires a dense deployment of access points, severely restricting the flexibility and placement of sensor nodes. GSM, on the other hand, requires costly data subscriptions and consumes significant

energy during transmission, substantially reducing the lifespan of battery-powered outdoor nodes.

To address these physical and technological barriers, this work focuses on three primary objectives:

- 1) **Long-range, low-power communication:** Deploying LoRa (Long Range) modulation at the 433MHz ISM band to achieve kilometer-scale communication range with minimal energy expenditure.
- 2) **Hardware and energy optimization:** Designing an "ESP-Only" architecture that bypasses traditional Arduino microcontrollers, combined with a strict hardware-software Deep Sleep mechanism to extend the outdoor Sensor Node's battery life.
- 3) **Continuous data accessibility through Edge AI:** Deploying a Gateway Node that hosts an embedded Local Web Server. This allows farmers and researchers to access Edge AI insights with zero latency (e.g., localized weather forecasting, pump control) even during Internet outages, while still supporting asynchronous synchronization to the Cloud via ThingSpeak.

II. LITERATURE REVIEW AND RESEARCH GAP

The foundation of this project builds on a comprehensive review of recent literature on Wireless Sensor Networks (WSN) for agricultural monitoring.

A. Comparison of Wireless Protocols

Prior studies have extensively compared Wi-Fi, Zigbee, and LoRa. Wi-Fi, while suitable for high-speed, real-time greenhouse monitoring, suffers from severe range limitations and high energy drainage. Zigbee (IEEE 802.15.4) offers a low-power alternative and is widely used in mesh networks; however, its short transmission range requires deploying numerous intermediate router nodes, complicating network topology and increasing deployment cost for large-scale farms.

In contrast, LoRa and Low-Power Wide Area Networks (LPWAN) represent a paradigm shift. LoRa's Chirp Spread Spectrum (CSS) modulation provides exceptional resilience to multipath fading and Doppler shifts, allowing it to penetrate dense foliage and achieve transmission ranges of several kilometers. Since meteorological data (temperature, humidity, etc.) involves small payloads and does not require high bandwidth, LoRa's low data rate is not a limitation but rather an ideal fit for long-term environmental monitoring.

B. Identified Research Gaps

Analysis of existing LoRa-based weather station prototypes reveals two notable research gaps:

- **Redundant hardware architectures:** Many proposed systems adopt a hybrid architecture combining an Arduino UNO (for sensor data acquisition) with an ESP8266/ESP32 (for network transmission). This redundancy increases BOM cost, expands the physical circuit footprint, and drains battery power unnecessarily. A typical example is systems combining an Arduino Uno and a NodeMCU for smart soil-moisture monitoring and irrigation [11].
- **Absolute reliance on cloud infrastructure:** Most existing prototypes merely act as data forwarders to third-party cloud servers (e.g., ThingSpeak, AWS). If the Internet connection at the Gateway is interrupted, the entire monitoring system becomes useless, depriving local users of critical real-time alerts.

The proposed system directly addresses both gaps by implementing a streamlined ESP-Only architecture and a local Edge AI Dashboard, ensuring continuous operation regardless of external Internet connectivity.

III. SYSTEM ARCHITECTURE

The project adopts a distributed Master-Slave IoT model, split into two geographically and functionally distinct components: the outdoor Sensor Node (transmitter) and the indoor Gateway Node (receiver).

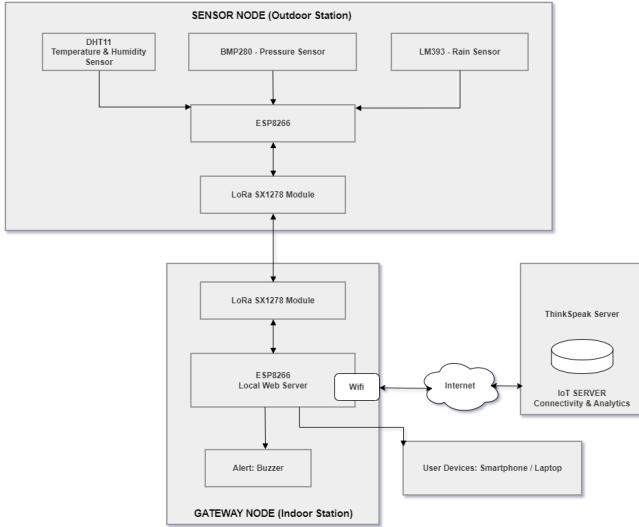


Fig. 1: Block diagram of the ESP-Only Master-Slave architecture and LoRa P2P communication link.

A. Sensor Node (Outdoor Transmitter)

The Sensor Node is designed to operate autonomously in harsh outdoor environments, powered by a rechargeable battery or a small solar panel. Its main function is to collect environmental parameters and transmit them over long distances. The node comprises:

- **Microcontroller:** NodeMCU ESP8266 (ESP-12E).
- **Sensors:**
 - *DHT11*: Measures ambient air temperature and relative humidity.
 - *BMP280*: High-precision I2C sensor measuring atmospheric pressure (used for weather forecasting).
 - *Analog rain sensor (LM393)*: Captures rainfall intensity through an analog surface-conductivity reading, rather than a simple binary digital signal.
- **Communication:** Semtech SX1278 LoRa module interfaced via the SPI bus.

B. Gateway Node (Indoor Receiver)

The Gateway Node serves as the central processing hub. Positioned indoors with mains power (5V adapter) and an Internet-connected Wi-Fi router, it bridges the LoRa network with IP-based networks.

- **Microcontroller:** NodeMCU ESP8266 configured in dual mode (WIFI_AP_STA).
- **Communication:** Semtech SX1278 LoRa module receiving packets via SPI.
- **Alerting/Actuation:** A physical buzzer for immediate acoustic alarms upon detecting weather anomalies.
- **Data management:** Hosts an embedded web server for local monitoring while performing HTTP GET requests to synchronize data to the ThingSpeak platform.

IV. HARDWARE DESIGN AND ENERGY OPTIMIZATION

Hardware optimization is at the core of this project. By transitioning to an ESP-Only design, redundant analog-to-digital converters and secondary microcontrollers are eliminated entirely.

A. Detailed Wiring / Pin Mapping

To ensure hardware reproducibility, Table I and Table II present the detailed pin configuration between the ESP8266 microcontroller and the peripheral components at each node.

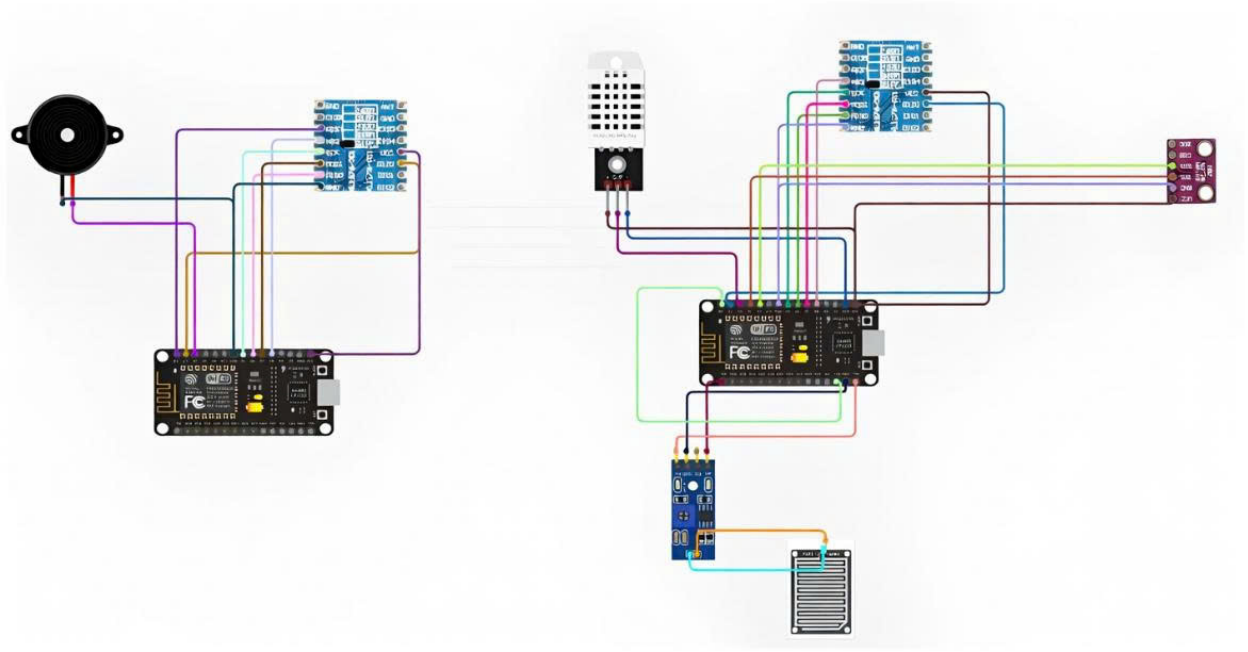


Fig. 2: Detailed circuit schematic for both the Sensor Node and the Gateway Node, showing SPI connections and the physical Deep Sleep D0-RST wire.

TABLE I: Wiring Table – Outdoor Sensor Node (ESP8266)

Module	Pin	ESP8266 Pin
LoRa SX1278	NSS (CS)	D8
	DIO0	D1
	SCK	D5
	MISO	D6 (default SPI)
	MOSI	D7 (default SPI)
	VCC	3.3V
	GND	GND
DHT11	DATA	D2 (GPIO4)
	VCC	3.3V
	GND	GND
BMP280	SDA	D4 (GPIO2)
	SCL	D3 (GPIO0)
	VCC	3.3V
	GND	GND
Rain Sensor	Analog (A0)	A0
	VCC	3.3V
	GND	GND
Deep Sleep	D0	RST (connect only after successful code upload)

TABLE II: Wiring Table – Indoor Gateway Node (ESP8266)

Module	Pin	ESP8266 Pin
LoRa SX1278	NSS (CS)	D8 (GPIO15)
	DIO0	D1 (GPIO5)
	RST	D0 (GPIO16)
	SCK	D5
	MISO	D6
	MOSI	D7
	VCC	3.3V
	GND	GND
Buzzer	Signal (+)	D2
	Ground (–)	GND

B. System Bill of Materials

Table III summarizes the main components and peripheral devices used throughout the system.

TABLE III: System Components and Peripheral Devices

Component / Device	Model / Notes
Sensor Node MCU	NodeMCU ESP8266 (ESP-12E)
Gateway Node MCU	NodeMCU ESP8266 (AP/STA)
Temperature & humidity sensor	DHT11
Pressure sensor	BMP280 (I2C)
Rain sensor	LM393, analog
LoRa module (x2)	Semtech SX1278, 433MHz
Alerting	Buzzer
Sensor Node power	Rechargeable battery
Gateway Node power	5V power adapter

C. Hardware Deep Sleep Implementation

To maximize the battery life of the outdoor Sensor Node, the team implemented the ESP8266's Deep Sleep mode. In this state, the CPU, Wi-Fi modem, and peripherals are powered down, reducing current draw to the micro-ampere (μA) range. However, for the ESP8266 to autonomously wake up after a predefined interval (e.g., 60 seconds), a physical hardware intervention is required: a jumper wire connecting GPIO16 (D0) to the RST (Reset) pin. This allows the internal Real-Time Clock (RTC) to issue a wake-up pulse.

V. SOFTWARE IMPLEMENTATION AND ALGORITHMS

The software algorithms for both nodes are carefully designed to ensure data integrity, network security, and compliance with API rate limits.

A. Sensor Node Algorithm

The Sensor Node operates on a strictly event-driven cycle to minimize active radio Time-on-Air.



Fig. 3: Algorithm flowchart of the Sensor Node (outdoor transmitter).

Upon waking, the node initializes the I2C and SPI buses. To prevent false positives caused by dew or condensation, the analog rain sensor undergoes a noise-filtering loop, averaging 16 consecutive `analogRead()` samples.

Additionally, to optimize LoRa transmission, sending concatenated ASCII strings (e.g., "T:30.5,H:80.0") was rejected due to high byte overhead and heap memory fragmentation. Instead, a custom 15-byte packed binary struct named `WeatherPkt_t` was designed. This payload encapsulates the Node ID, sequence number, compressed sensor data, battery percentage, and a CRC16 checksum for data validation over the noisy 433MHz spectrum. Before entering Deep Sleep, the firmware sends a direct SPI command to the SX1278 (`REG_OP_MODE`) to force the LoRa module into sleep mode, preventing a 1.5mA standby current leak.

B. Gateway Node Algorithm

The Gateway Node must handle asynchronous tasks: listening for incoming LoRa packets, maintaining a local Wi-Fi Access Point, and communicating with the Cloud.

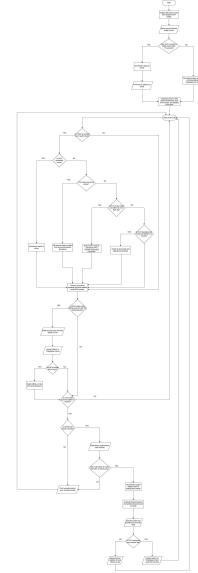


Fig. 4: Algorithm flowchart of the Gateway Node (indoor receiver).

To prevent packet sniffing and cross-network interference from other 433MHz devices, the Gateway and Sensor nodes are cryptographically bound using a predefined Synchronization Word (`LoRa.setSyncWord(0xAB)`). When a valid packet passes the CRC check, the Gateway decodes the binary data. If an anomaly is detected (e.g., temperature $> 40^{\circ}C$ or rain-sensor analog value < 450), a GPIO pin triggers the local buzzer.

For cloud synchronization, the firmware strictly avoids blocking functions such as `delay()`. Instead, an asynchronous `millis()`-based timer schedules periodic HTTP GET requests to the ThingSpeak API, safely respecting the free-tier rate limit while keeping the local web server responsive.

VI. SYSTEM INTEGRATION: CLOUD DATA TIERING AND LOCAL EDGE FORECASTING

This section details the multi-tiered data pipeline of the project, including the binary packet compression scheme at the sensor node, integration with a cloud-assisted machine-learning forecasting service, and edge-computing algorithms used to optimize communication bandwidth.

A. LoRa Transmission Protocol and Binary Packing

To minimize radio airtime occupied by the Semtech SX1278 module on the unlicensed 433 MHz band, the project eliminates cumbersome ASCII strings (JSON/XML) entirely. Sensor data is instead packed directly into a minimal 15-byte binary struct:

```
#pragma pack(push, 1)
typedef struct {
    uint16_t node_id;
    uint8_t seq_num;
    uint8_t flags;
    int16_t temperature; // x100
    uint16_t humidity; // x100
    uint16_t pressure; // -800, x10
    uint16_t rain_analog; // 0-1023
    uint8_t battery_pct;
    uint16_t crc16;
} WeatherPkt_t;
#pragma pack(pop)
```

Floating-point values are cast and compressed into integers at the edge to reduce per-parameter storage size using the following scaling expressions:

$$T_{\text{int}} = \lfloor T_{\text{float}} \times 100 \rfloor \quad (1)$$

$$H_{\text{int}} = \text{clamp}(H_{\text{float}} \times 100, 0, 10000) \quad (2)$$

$$P_{\text{int}} = \text{clamp}(P_{\text{float}} - 800.0, 0, 300) \times 10 \quad (3)$$

This packing scheme limits radio Time-on-Air to approximately 28.8 ms (at SF7, 125 kHz bandwidth configuration), helping the outdoor battery-powered node maximize energy savings.

B. Cloud Channel Configuration and Data Stream Synchronization

After decoding a binary packet and successfully verifying its CRC-16 checksum, the Gateway pushes the measured data to the ThingSpeak cloud. To prevent data overwrite issues that could distort the real-time Dashboard, the data channel is tiered as follows: (i) the real measurement stream (Fields 1–5), written directly by the Gateway from the local LoRa sensor network; and (ii) the AI forecast stream (Fields 6–8), asynchronously updated every 15 minutes by a meteorological analytics server (ML Engine), as summarized in Table IV.

TABLE IV: ThingSpeak Channel Field Allocation

Field	Parameter	Source
Field 1	Temperature (°C)	Gateway (edge)
Field 2	Humidity (%)	Gateway (edge)
Field 3	Pressure (hPa)	Gateway (edge)
Field 4	Rain index (0–1023)	Gateway (edge)
Field 5	Sensor node battery (%)	Gateway (edge)
Field 6	+1h temperature forecast	ML Engine
Field 7	Rain probability	ML Engine
Field 8	+1h humidity forecast	ML Engine

C. Cloud-Assisted AI Forecasting Integration

To help users make smart irrigation decisions, the system connects to an AI forecasting server running on an automated cloud service. The forecasting model uses a Random Forest algorithm trained on three years of historical data from the Southern Vietnam region, enabling a 1-hour-ahead forecast of temperature, humidity, and rain probability. The system achieves an experimental MAE of 0.34°C for temperature and 2.12% for humidity.

To improve accuracy during rapidly forming storm clouds in the field, the system applies a hybrid correction mechanism at the edge. If the rain sensor reading exceeds a threshold ($R_{\text{analog}} < 700$), the Gateway automatically overrides the local rain probability:

$$P_{\text{rain}} = \max(P_{\text{rain}}, 60\% + (700 - R_{\text{analog}}) \times 0.05) \quad (4)$$

D. Edge Fallback Linear Regression (Edge Computing)

To ensure high availability, the project implements an edge-computing algorithm using an ordinary least squares (OLS) linear regression model that runs directly on the Gateway or the local Dashboard interface when Internet connectivity is lost.

The algorithm continuously computes weather trends based on N locally stored historical data points (with the origin set at the current measurement time $t = 0$):

$$a = \frac{N \sum_{i=1}^N (t_i \cdot Y_i) - \sum_{i=1}^N t_i \sum_{i=1}^N Y_i}{N \sum_{i=1}^N t_i^2 - \left(\sum_{i=1}^N t_i \right)^2} \quad (5)$$

$$b = \bar{Y} - a \cdot \bar{t} \quad (6)$$

Temperature, humidity, and pressure for the next +60-minute cycle are quickly extrapolated at the edge using:

$$\hat{Y}(t + 60\text{m}) = a \cdot 60 + b \quad (7)$$

E. Data Transmission Protection and Compensating Synchronization

To keep the system operating stably even under an unreliable field Wi-Fi connection, the Gateway implements two embedded software protection mechanisms:

- 1) **Rate-Limit Guard:** To comply with ThingSpeak's 15-second-per-update limit, the Gateway filters outgoing data



Fig. 5: Long-term historical data analytics dashboard on the ThingSpeak cloud.

using an asynchronous time comparison $\Delta t_{\text{last_send}} \geq 20$ seconds, avoiding API rejection errors from the cloud server.

- 2) **Local buffering and compensating synchronization:** When network connectivity is lost, the Gateway automatically switches to writing data into a temporary file `/offline.csv` on the ESP8266's flash memory via the LittleFS library. Once Wi-Fi is restored, an asynchronous background task automatically reads this file and re-sends each buffered record to the Cloud with a safe 16-second interval, preserving the integrity of the historical data series.

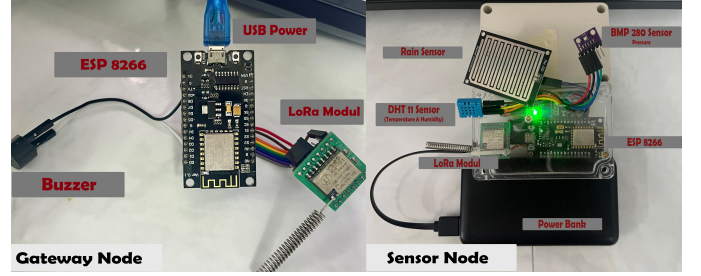


Fig. 7: Assembled system prototype (Sensor Node and Gateway Node).

VII. EXPERIMENTAL RESULTS AND DISCUSSION

A. Prototype Implementation

The system prototype was fully assembled based on the block diagram and circuit design presented in Sections III and IV, using a breadboard and the components listed in Table III.

B. Field Test Setup

The Sensor Node was placed outdoors to evaluate real-world data-collection performance, while the Gateway Node remained indoors, connected to a Wi-Fi router and mains power.

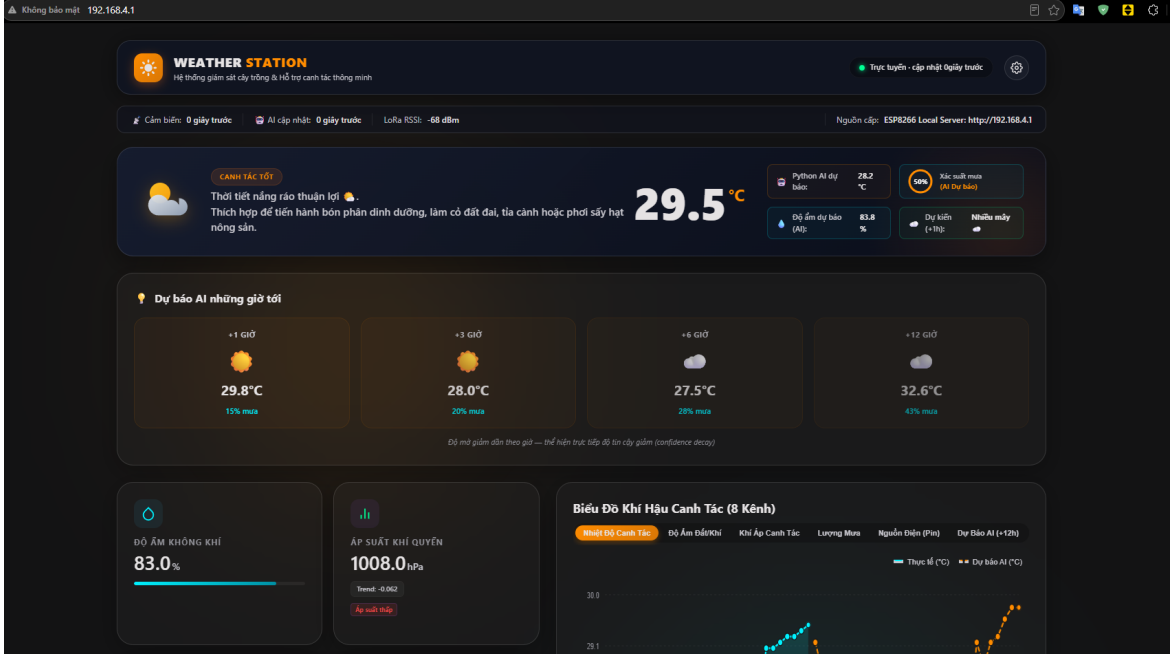


Fig. 6: Local Edge Web Dashboard interface for offline monitoring at IP address 192.168.4.1.



Fig. 8: Sensor Node deployed for outdoor field testing.

Key results and observations recorded during testing include:

- The local Edge AI Dashboard successfully recorded and displayed real-time data (e.g., 31.5°C, 78% humidity, 1009 hPa pressure) along with a continuous 24-hour temperature chart, all without requiring an Internet connection.
- The ThingSpeak channel successfully logged more than 150 data entries, confirming that the asynchronous synchronization mechanism operates stably and remains within the free-tier rate limit.
- The CRC-16 verification mechanism effectively discarded packets corrupted by 433MHz-band noise, ensuring data integrity prior to decoding and cloud upload.
- The Random Forest forecasting model achieved a test-set MAE of 0.34°C (temperature) and 2.12% (humidity).

C. Field Measurement Data

Table V presents a portion of the actual telemetry data recorded at the Gateway during the test session on July 08, 2026.

TABLE V: Telemetry Data Recorded at the Gateway (July 08, 2026)

Time	T (°C)	H (%)	P (hPa)	Rain	RSSI
18:00	31.75	71.00	1008.50	1024	-59
19:00	31.70	72.69	1009.10	1024	-64
20:00	32.34	71.50	1009.90	1024	-57
21:00	32.06	71.50	1010.50	1024	-60
22:00	32.02	71.50	1010.90	1024	-62

VIII. CONCLUSION

This work presented and successfully implemented an energy-efficient, LoRa-based smart weather station integrated with Edge AI. The ESP-Only architecture, combined with 15-byte binary payload compression and a hardware Deep Sleep mechanism, effectively resolves the trade-off between transmission range, power consumption, and hardware cost that persists in traditional designs combining Arduino with ESP8266. The tiered data approach, pairing a local Edge AI Dashboard that operates independently of the Internet with a machine-learning forecasting service on the ThingSpeak cloud platform, ensures that critical weather data and alerts remain accessible under all network conditions.

Field test results confirm stable system operation: the CRC-16 verification mechanism effectively discards packets corrupted by noise, the compensating synchronization mechanism

prevents data loss during network outages, and the machine-learning forecasting model achieves high accuracy (MAE of 0.34°C for temperature). These results demonstrate the feasibility of deploying a low-cost, energy-efficient weather monitoring system capable of operating independently in agricultural areas lacking stable network infrastructure.

Building on these results, several promising directions for future research include: (i) scaling the sensor network to a many-transmitter, one-receiver (star) topology to cover larger monitored areas; (ii) incorporating periodic calibration for the DHT11/BMP280 sensors to reduce long-term measurement drift; and (iii) integrating renewable energy sources (solar panels) with smart charge management to eliminate the need for periodic battery replacement altogether.

REFERENCES

- [1] G. R. Mendez, M. A. M. Yunus, S. C. Mukhopadhyay, "A Wi-Fi based Smart Wireless Sensor Network for Monitoring an Agricultural Environment," *IEEE Sensors Journal*, 2012.
- [2] A. Pagano et al., "A Survey on LoRa for Smart Agriculture: Current Trends and Future Perspectives," *Computers and Electronics in Agriculture*, 2023.
- [3] H. M. Jawad et al., "Energy-Efficient Wireless Sensor Networks for Precision Agriculture: A Review," *Sensors*, vol. 17, no. 8, p. 1781, 2017.
- [4] D. Sumanth, G. Bharadwaja, B. V. K. Reddy, S. K. Satyanarayana, "LoRa based Wireless Weather Station," *IJRASET*, vol. 9, no. 6, pp. 1214–1220, Jun. 2021.
- [5] F. R. Saputri et al., "Design and development of an irrigation monitoring and control system based on blynk internet of things and thingspeak," *PLOS ONE*, vol. 20, no. 4, e0321250, 2025.
- [6] I. Michailidis et al., "An Arduino-Based, Portable Weather Monitoring System, Remotely Usable Through the Mobile Telephony Network," *MDPI Electronics*, 2025.
- [7] M. Artiyasa et al., "Comparative Study of Internet of Things (IoT) Platform for Smart Home Lighting Control Using NodeMCU with Thingspeak and Blynk Web Applications," *Fidelity: Jurnal Teknik Elektro*, vol. 2, no. 1, pp. 1–6, 2020.
- [8] N. M. Yoeseph et al., "Prototype weather station uses LoRa wireless connectivity infrastructure," *Journal of Physics: Conference Series*, vol. 1367, p. 012089, 2019.
- [9] S. Ganesan et al., "IoT-enabled Smart Weather Stations: Innovations, Challenges, and Future Directions," *Malaysian Journal of Science and Advanced Technology*, vol. 4, no. 2, pp. 180–190, 2024.
- [10] J. R. Cortes León et al., "Raspberry Pi and Arduino UNO Working Together as a Basic Meteorological Station," *IJCSIT*, vol. 9, no. 5, pp. 97–104, 2017.
- [11] M. K. Kankara, A. Imtiaz, I. Chowdhury, Md. K. M. Khan, T. Ahmed, "Arduino and NodeMCU-Based Smart Soil Moisture Balancer with IoT Integration," in *Information Systems for Intelligent Systems*, Smart Innovation, Systems and Technologies, vol. 324, Springer, Singapore, 2023, pp. 621–636.
- [12] H. D. Thinh et al., "Mini Literature Review: LoRa-Based Weather Station Monitoring System," FPT University Project Report, Summer 2026.